🎓 جزوه طلایی سیستم‌عامل

فصل ۱۰: حافظه مجازی - بخش ششم (تخصیص قاب‌ها و کوبیدگی)

۱. محدودیت‌های تخصیص قاب (Frame Allocation Constraints)

ما نمی‌توانیم هر تعداد دلمان خواست به پردازه‌ها قاب بدهیم. دو محدودیت داریم:

- **حداکثر (Maximum):** محدود به کل حافظه فیزیکی (RAM) سیستم.
- **حداقل (Minimum):** محدود به معماری کامپیوتر.
  - **چرا حداقل داریم؟** اگر خطای صفحه رخ دهد، دستور باید **تکرار (Restart)** شود. پس باید حداقل تعداد قابی که برای اجرای یک *دستور واحد* نیاز است را داشته باشیم.
  - *مثال:* اگر دستوری دارای آدرس‌دهی غیرمستقیم (Indirect Addressing) باشد، ممکن است به ۳ قاب نیاز داشته باشد (یکی برای خود دستور، دو تا برای آدرس‌ها). اگر فقط ۲ قاب بدهیم، دستور هرگز اجرا نمی‌شود و در حلقه بی‌نهایت خطای صفحه می‌افتد!

۲. الگوریتم‌های تخصیص (Allocation Algorithms)

چطور m قاب موجود را بین n پردازه تقسیم کنیم؟

الف) تخصیص مساوی (Equal Allocation)

- به همه سهم برابر می‌دهیم: m/n.
- **عیب:** عدالت کورکورانه! یک برنامه نوت‌پد ۱۰ کیلوبایتی همان‌قدر رم می‌گیرد که دیتابیس ۱ گیگابایتی می‌گیرد (اتلاف حافظه).

ب) تخصیص متناسب (Proportional Allocation)

- بر اساس **اندازه** پردازه سهم می‌دهیم. هر چه پردازه بزرگتر، سهم بیشتر.
- **فرمول:** اگر S مجموع اندازه تمام پردازه‌ها و m کل قاب‌ها باشد، سهم پردازه p_i برابر است با:

- $$a_i = \frac{s_i}{S} \times m$$

۳. تخصیص سراسری در برابر محلی (Global vs. Local Allocation)

وقتی یک پردازه نیاز به صفحه جدید دارد و رم پر است، قربانی را از کجا انتخاب کنیم؟

ویژگی	تخصیص سراسری (Global)	تخصیص محلی (Local)
روش انتخاب	می‌تواند قاب هر کسی (حتی بقیه پردازه‌ها) را بدزدد.	فقط اجازه دارد قاب‌های خودش را جایگزین کند.
مزیت	**بازدهی (Throughput) بالاتر** (رایج‌تر است).	عملکرد **قابل پیش‌بینی** و پایدار برای هر پردازه.
عیب	عملکرد یک پردازه به رفتار بقیه وابسته می‌شود (اگر بقیه رم‌خوار باشند، این کند می‌شود).	ممکن است حافظه هدر برود (یکی رم اضافه دارد، دیگری کم دارد ولی نمی‌توانند قرض بگیرند).

⚠ نکته امنیتی لینوکس (OOM Killer):

اگر در تخصیص سراسری رم به شدت کم بیاید، لینوکس وارد عمل شده و با **Out-Of-Memory (OOM) Killer** یکی از پردازه‌ها را می‌کشد تا رم آزاد شود! (معمولاً پردازه‌ای که بیشترین رم را گرفته).

۴. انواع خطای صفحه (Major vs. Minor Page Faults)

همه خطاهای صفحه مساوی نیستند:

خطای اصلی (Major / Hard Fault):

- صفحه در رم نیست و باید از **دیسک** خوانده شود.
- **هزینه:** بسیار زیاد (کندی شدید).

خطای فرعی (Minor / Soft Fault):

- صفحه در دیسک نیست، بلکه جای دیگری در **رم** است اما پردازه ما به آن دسترسی (Mapping) ندارد.
- *مثال ۱:* اشتراک کتابخانه (Library) که قبلاً توسط برنامه دیگری لود شده.
- *مثال ۲:* صفحه‌ای که آزاد شده اما هنوز محتوایش پاک نشده (Reclaim).
- **هزینه:** بسیار کم (فقط آپدیت جدول صفحه).

۵. دسترسی حافظه غیریکپارچه (NUMA)

در سیستم‌های چندپردازنده‌ای مدرن، سرعت دسترسی CPU به حافظه یکسان نیست.

- **NUMA (Non-Uniform Memory Access):** هر CPU یک حافظه محلی دارد که به آن سریع‌تر دسترسی دارد تا حافظه متصل به CPU دیگر.
- **هدف مدیریت حافظه:** قاب‌هایی را به پردازه تخصیص دهیم که **نزدیک‌ترین** فاصله فیزیکی را به CPU اجراکننده آن پردازه داشته باشند (بهبود Latency).
- *سولاریس:* از lgroups (گروه‌های محلی) برای حل این مشکل استفاده می‌کند.

🚨 ۶. کوبیدگی (Thrashing)

بدترین کابوس سیستم‌عامل!

- **تعریف:** وقتی پردازه زمان بیشتری را صرف **صفحه‌بندی (Paging)** کند تا صرف **اجرای دستورات**.
- **علت:** پردازه قاب‌های کافی برای نگهداری «مجموعه کاری» (Working Set) خود ندارد.
- **چرخه مرگبار:**
  1. رم کم می‌آید ⬅ خطای صفحه زیاد می‌شود.
  2. CPU بیکار می‌شود (چون همه منتظر دیسک هستند).
  3. سیستم‌عامل فکر می‌کند CPU بیکار است ⬅ درجه چندبرنامگی را زیاد می‌کند (برنامه جدید باز می‌کند).
  4. برنامه جدید رم می‌خواهد ⬅ وضعیت بدتر می‌شود ⬅ **سیستم قفل می‌کند.**

📝 خلاصه نکات طلایی بخش ششم:

1. **حداقل قاب:** محدودیت معماری (برای Restart دستورات).
2. **تخصیص متناسب:** عدالت بر اساس سایز برنامه (s_i/S).
3. **Global Replacement:** دزدی از همسایه! (سریع‌تر اما غیرقابل پیش‌بینی).
4. **Minor Fault:** صفحه در رم هست، فقط آدرس‌دهی می‌خواهد (سریع).
5. **NUMA:** رمِ نزدیک به CPU را بدهید.
6. **Thrashing:** وقتی سرعت پیجینگ از سرعت اجرا بیشتر شود (مرگ سیستم).